MADUREZ Y ESTANDARES EN PROYECTOS TI

J. J. Chán Cuellar

7690-22-1805 Universidad Mariano Gálvez

Seminario de Tecnologías de Información

jchanc4@miumg.edu.gt

Resumen

Este trabajo tuvo como objetivo analizar las prácticas que permiten garantizar la calidad de un proyecto de Tecnologías de la Información (TI). Para ello, se revisaron fuentes sobre modelos de calidad de software, aseguramiento y control de calidad, gestión de requisitos, pruebas y mejora de procesos. Como resultado, se identificó que la calidad abarca la seguridad, la facilidad de uso, la confiabilidad y la capacidad de responder a las necesidades de los usuarios, y no solo el funcionamiento técnico. Se determinó que las prácticas de mayor impacto son definir requisitos y criterios de aceptación desde el inicio, aplicar pruebas constantes, controlar los cambios, proteger la información, mantener la documentación y involucrar a los usuarios. Asimismo, se observó que indicadores como la cantidad de defectos y la cobertura de pruebas permiten tomar decisiones con base en resultados concretos. Se concluye que la calidad debe considerarse en todas las etapas del proyecto y requiere un esfuerzo coordinado entre responsables, desarrolladores y usuarios.

Palabras claves: calidad de software, proyectos de TI, pruebas de software, mejora continua, aseguramiento de la calidad.

Desarrollo del tema

Garantizar la calidad de un proyecto de Tecnologías de la Información (TI) no consiste únicamente en comprobar que un programa funcione correctamente al momento de entregarlo. También es importante que sea seguro, fácil de utilizar, confiable y capaz de responder a las necesidades de los usuarios. Un sistema puede funcionar sin errores de programación, pero presentar dificultades si es lento, complicado de utilizar o no cumple con lo que necesita la organización. Por ello, la calidad debe tomarse en cuenta desde la planificación hasta la implementación y el mantenimiento. La norma ISO/IEC 25010 (de la International Organization for Standardization y la International Electrotechnical Commission) establece en su versión de 2023 nueve características para evaluar la calidad del producto de software, entre ellas la adecuación funcional, la fiabilidad, la capacidad de interacción, la seguridad y la mantenibilidad (ISO/IEC, 2023). Esto demuestra que un proyecto exitoso debe ofrecer una solución útil y confiable, no solamente cumplir con los aspectos técnicos.
Para lograrlo, es necesario definir claramente los requisitos desde el inicio. Según el Project Management Institute (PMI, 2021), la gestión de la calidad implica planificar y comprobar que los resultados cumplan con lo esperado. Para ello, se puede elaborar una Especificación de Requerimientos de Software (ERS), donde se describan las funciones del sistema y sus condiciones de funcionamiento, como el rendimiento y la seguridad. También es útil establecer criterios de aceptación y matrices de trazabilidad que permitan relacionar cada requisito con las funciones desarrolladas y las pruebas correspondientes. Esto ayuda a evitar malentendidos, detectar requisitos pendientes y reducir cambios costosos en etapas avanzadas del proyecto.

Es importante poder distinguir entre el aseguramiento de la calidad (QA) y el control de calidad (QC). El QA busca prevenir errores mediante buenas prácticas y procesos organizados, mientras que el QC se enfoca en identificar defectos a través de revisiones y pruebas. Chemuturi (2011) destaca la importancia de gestionar la calidad de manera organizada. Por ejemplo, un equipo puede revisar la seguridad del sistema, comprobar si soportará la cantidad de usuarios esperada y verificar que sus componentes funcionen correctamente al integrarse. Identificar estos riesgos con anticipación permite corregir problemas antes de que afecten a los usuarios.

Las pruebas de software también son fundamentales para comprobar que el sistema funcione correctamente en diferentes situaciones. Estas pueden incluir pruebas unitarias, de integración y de aceptación. Además, herramientas como pytest, para aplicaciones desarrolladas con Python, permiten automatizar verificaciones y comprobar que los cambios realizados no provoquen errores en funciones que anteriormente trabajaban bien. Aunque las pruebas automatizadas no sustituyen por completo las revisiones manuales, ayudan a ahorrar tiempo y detectar fallos con mayor rapidez.

Es importante controlar los cambios mediante herramientas como Git, que permiten mantener un historial de las modificaciones realizadas en el código. Las revisiones entre desarrolladores también ayudan a identificar errores y mantener buenas prácticas de programación. Para evaluar los resultados, se pueden utilizar indicadores como la cantidad de defectos encontrados, la cobertura de las pruebas y los tiempos de respuesta del sistema. Estos datos permiten conocer qué aspectos necesitan mejoras y tomar decisiones basadas en resultados concretos.

No basta con realizar pruebas; también es necesario prestar atención a la seguridad de la información. Los sistemas pueden manejar datos personales, información financiera o documentos internos que deben protegerse contra accesos no autorizados. Por esta razón, es importante establecer permisos de acceso, utilizar mecanismos de autenticación y mantener actualizados los componentes del software. También se deben realizar copias de seguridad y comprobar que la información pueda recuperarse ante una falla. Estas medidas ayudan a reducir los riesgos y a mantener la confianza de los usuarios en el sistema.

La calidad también depende de la organización y de la forma en que trabaja el equipo. En este sentido, el modelo Capability Maturity Model Integration (CMMI) ofrece una referencia para evaluar y mejorar los procesos relacionados con el desarrollo de software (CMMI Institute, 2020). Un proyecto final de graduación de la Universidad Nacional de Costa Rica propone que la función de calidad se establezca de forma institucional dentro de la dirección de TI (Pacheco Aviles, 2017). Aplicar procedimientos claros, distribuir responsabilidades y documentar las actividades permite mantener una forma de trabajo organizada y evita que los resultados dependan únicamente de la experiencia de un desarrollador. Asimismo, realizar reuniones periódicas para revisar los avances permite identificar retrasos, resolver dudas y corregir problemas antes de que afecten el cumplimiento de los objetivos.

Es indispensable mantener una comunicación constante con los usuarios y permitirles probar el sistema durante su desarrollo. Esto ayuda a comprobar si las funciones cumplen con sus necesidades, si la interfaz es fácil de utilizar y si existen dificultades que deban corregirse antes de la entrega. Los comentarios de los usuarios pueden revelar problemas que el equipo de desarrollo no había considerado, por lo que es importante tomarlos en cuenta y realizar los ajustes necesarios.

Se debe mantener actualizada la documentación del proyecto, incluyendo los manuales de usuario, los casos de prueba, las instrucciones técnicas y los registros de errores. Esta información facilita la solución de problemas y permite que otros integrantes del equipo comprendan cómo funciona el sistema. De igual manera, es recomendable evaluar el software después de su implementación para comprobar si continúa respondiendo adecuadamente a las necesidades de la organización y detectar

posibles mejoras.
Un punto clave es la mejora continua, ya que la calidad no termina cuando el sistema se entrega. Con el tiempo pueden surgir nuevas necesidades, cambiar los requisitos de la organización o aparecer problemas que no se detectaron durante las primeras pruebas. Por ello, es conveniente revisar periódicamente el funcionamiento del software, analizar los errores reportados y determinar qué cambios pueden mejorar su rendimiento y seguridad. Este seguimiento permite mantener el sistema actualizado y evitar que pequeños problemas se conviertan en dificultades mayores.
Para que todas estas prácticas den buenos resultados, es necesario que exista un compromiso compartido entre los responsables del proyecto, los desarrolladores y los usuarios. Cada integrante debe cumplir con sus responsabilidades y comunicar oportunamente cualquier dificultad que pueda afectar el resultado. Cuando el equipo trabaja de manera coordinada, sigue procedimientos claros y utiliza los resultados de las pruebas para tomar decisiones, es más fácil cumplir con los requisitos y aprovechar mejor los recursos disponibles.
Garantizar la calidad de un proyecto de TI requiere planificación, prevención de riesgos, pruebas constantes, seguridad, control de cambios y participación de los usuarios. Cuando estos elementos se aplican durante todo el desarrollo, es posible reducir errores y obtener una solución segura, confiable y útil a largo plazo. La calidad no debe considerarse una tarea exclusiva de los programadores ni una revisión que se realiza al finalizar el proyecto, sino una responsabilidad compartida por todo el equipo para asegurar que la tecnología realmente cumpla con su propósito.

Observaciones y comentarios

Las prácticas revisadas son útiles, pero su aplicación depende del tamaño y la madurez de cada equipo. Modelos como CMMI pueden resultar costosos para proyectos pequeños, por lo que conviene priorizar los procesos de mayor impacto, como los criterios de aceptación y las pruebas. Además, las pruebas automatizadas reducen el tiempo de verificación, pero no detectan problemas de usabilidad, que solo se identifican con la participación de los usuarios.

Conclusiones

1. La calidad de un proyecto de TI requiere una planificación adecuada y la definición clara de los requisitos desde el inicio.

2. La comunicación y la distribución de responsabilidades facilitan el trabajo en equipo y el cumplimiento de los objetivos.

3. La calidad debe mantenerse durante todo el ciclo de vida del proyecto mediante la evaluación de resultados y la mejora continua.

Bibliografía

Chemuturi, M. (2011). *Mastering software quality assurance: Best practices, tools and techniques for software developers*. J. Ross Publishing.

CMMI Institute. (2020). *CMMI V2.0: Capability Maturity Model Integration for Development*. ISACA.

ISO/IEC. (2023). *Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model* (ISO/IEC 25010:2023). International Organization for Standardization.

Pacheco Aviles, E. (2017). *Diseño de una estrategia para la implementación del aseguramiento de calidad en el desarrollo de soluciones tecnológicas, del Departamento de Tecnología de Información de la Corporación CEFA S.A.* [Proyecto final de graduación, Universidad Nacional de Costa Rica]. https://repositorio.una.ac.cr/server/api/core/bitstreams/c41cf67a-1366-45e8-9ba9-b2fef8a059a8/content

Project Management Institute. (2021). *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)* (7.ª ed.). PMI.

Repositorio del proyecto

El código fuente y el documento en LaTeX se encuentran disponibles en el siguiente repositorio de GitHub:
https://github.com/jonathanc4chan/FORO-ACADEMICO-6-.git